

CSAI321 – Computer Networks

Scenario-Based Case-Study Question Bank

31 Applied/Scenario Questions with Full Model Answers (Total: 290 Marks)

Format note: These are long-answer, case-study style questions – the kind that give a realistic scenario and end in a bolded instruction (“Explain...”, “Diagnose...”, “Critically evaluate...”) with marks shown in [brackets], matching typical contest/exam phrasing. A full model answer is provided directly below each question. Attempt each question in your own words first (aim to hit the bolded command word specifically), then compare against the model answer.

Contents

Section 1 – Internet Fundamentals & Cloud Networking	2
Section 2 – Packet Switching & Network Delay	3
Section 3 – Protocol Layers (OSI & TCP/IP)	5
Section 4 – Devices, Topologies & Cloud Infrastructure	6
Section 5 – HTTP, REST & Application Protocols	8
Section 6 – HTTPS, TLS & Network Security	10
Exam Strategy Notes	13

Section 1 – Internet Fundamentals & Cloud Networking

Q1. A user connects to an unsecure public Wi-Fi network at an airport and attempts to access an online banking portal. An attacker executing a Man-in-the-Middle attack intercepts the traffic and attempts to present a fraudulent certificate. In the given scenario, **explain how Transport Layer Security (TLS) protects the user.** [10 marks]

Model Answer:

TLS protects the user through its **certificate validation** step in the handshake. Even though the attacker's device physically sits in the traffic path, the browser still expects the bank's server to present a certificate signed by a **trusted Certificate Authority (CA)**. The attacker cannot obtain the bank's real private key, so any certificate they present (self-signed or fraudulent) will **not** chain back to a CA root that the browser/OS already trusts. Certificate validation therefore fails, the browser raises a "Not Secure" / connection warning, and the TLS handshake does not proceed to the key-exchange and encrypted-communication stage. Even if the user ignored the warning, TLS's **confidentiality** (symmetric session-key encryption) and **integrity** (tamper detection) protections mean the attacker cannot silently read or modify banking credentials in transit – unlike plain HTTP, which offers no such protection.

Q2. A logistics company operates its own data center with 30 physical servers but is evaluating a move to AWS. Their network engineer claims: "Moving to the cloud means concepts like load balancers and subnets will not apply anymore – everything becomes fundamentally different." **Evaluate whether this claim is accurate**, referencing specific AWS concepts and their physical-world equivalents. [10 marks]

Model Answer:

The claim is **inaccurate**. Cloud computing does not make traditional networking concepts obsolete – it changes how they are *instantiated*, not what they *are*. A load balancer in AWS still performs the same job (distributing traffic across servers) as a physical load balancer; a **VPC subnet** still partitions a Layer 3 network exactly like a traditional subnet, just configured via API calls/console clicks instead of manual wiring. An **Availability Zone** plays the same fault-isolation role a physically separate data center would. The company's 30 physical servers map conceptually onto EC2 **virtual instances**; copper cabling maps onto **virtualized network overlays**. So the engineer is right that *execution* changes (software-defined vs. manual), but wrong that the underlying concepts disappear.

Q3. A startup's mobile app taps a button and, in under a second, a personalized recommendation appears. A junior developer says "it's simple – the phone just talks to WiFi and the server sends the data back." **Critique this explanation** using the actual end-to-end request journey. [5 marks]

Model Answer:

The explanation oversimplifies what actually happens. In reality the request passes: phone (app) → WiFi/4G radio (Physical/Data Link) → local router → ISP → the network core (many routers, each forwarding hop-by-hop) → possibly an Edge Location/CDN cache → a load balancer at the destination → an application server, with a TLS handshake securing the connection and TCP ensuring reliable delivery – and the entire reverse path for the response. The "WiFi to server" framing skips the network core, addressing/routing (Layer 3), reliability (Layer 4), and security (TLS) entirely, all of which genuinely happen within that sub-second window.

Q4. A company deploys its flagship application in only ONE Availability Zone within the Mumbai Region to save costs. Six months later, a regional power-grid failure affects that single AZ for several

hours, and the application goes completely offline. **Diagnose the architectural mistake and propose a fix using correct AWS terminology.** [10 marks]

Model Answer:

Mistake: The application had no fault-tolerance because it relied on a single Availability Zone. An AZ is one or more discrete data centers with redundant power/networking, acting as an **isolated fault domain** – but that isolation only protects you if you actually use more than one AZ; a single-AZ deployment inherits that one AZ's failure completely.

Fix: Deploy the application across **multiple AZs** (multi-AZ deployment) within the same Region, with a **load balancer** distributing traffic across healthy instances in each AZ. If one AZ fails, traffic automatically continues to be served by instances in the other AZ(s), keeping the application online. For even stronger resilience against a Region-wide event, a secondary deployment in an entirely different **Region** could also be considered.

Section 2 – Packet Switching & Network Delay

Q5. A video-conferencing company is deciding between reserving dedicated bandwidth per call (circuit-switching-like behavior) versus using standard best-effort packet switching. Their CFO argues packet switching is "strictly better in every situation since it's what the whole Internet uses." **Critically evaluate this claim** and identify at least one scenario where reserved-bandwidth guarantees would be preferable. [10 marks]

Model Answer:

The claim is an overgeneralization. Packet switching wins on **efficiency** (statistical multiplexing lets many bursty users share links) and **resilience** (traffic reroutes around failures), which is why it underlies the modern Internet. However, it provides **no built-in guarantees**: under congestion, packets can experience variable queuing delay or be dropped, causing jitter/frozen video – unacceptable for some use cases. A scenario where reserved capacity is preferable: a **surgical telemedicine consultation** or a **live financial trading feed** that needs strictly predictable, jitter-free bandwidth – here the guaranteed (if sometimes "wasteful") nature of a dedicated/reserved circuit (or modern QoS-guaranteed virtual circuits) is worth the inefficiency, because a dropped/delayed packet could have serious real-world consequences.

Q6. An e-commerce platform reports that customers on a "high-speed" 1 Gbps fibre plan still experience slow page loads during flash sales, while customers on modest 50 Mbps connections at other times report no issues. Diagnostic logs show: Customer's home router (1 Gbps) → ISP access link (500 Mbps) → ISP backbone (10 Gbps) → a congested peering exchange link (20 Mbps) → platform's data center network (40 Gbps). **Calculate the effective end-to-end throughput and explain, with reference to bottleneck theory, why the customer's 1 Gbps plan is irrelevant here.** Also recommend where engineering effort should be focused. [15 marks]

Model Answer:

Throughput calculation: End-to-end throughput $\approx \min(1 \text{ Gbps}, 500 \text{ Mbps}, 10 \text{ Gbps}, 20 \text{ Mbps}, 40 \text{ Gbps}) = 20 \text{ Mbps}$ – the congested peering exchange link is the bottleneck.

Explanation: Per the bottleneck rule (throughput = minimum link rate along the path), the customer's local 1 Gbps home connection is far from being the limiting factor; it is simply not the slowest link in the chain, so upgrading it further would yield *zero* improvement in this case. This is the classic "I pay for 300 Mbps, why is it buffering?" scenario generalized – one slow hop anywhere on the path caps the whole path's throughput regardless of how fast the other hops are.

Recommendation: Engineering effort should focus specifically on the **peering exchange link**

– e.g., provisioning additional peering capacity, adding more peering points, or using a CDN/Edge Location to bypass that congested link entirely during flash sales, rather than telling customers to “upgrade their internet.”

Q7. During a network audit, a router is found to occasionally forward only PART of a large packet before its outbound link temporarily fails, then resumes sending the remaining bits once the link recovers, claiming this improves latency. **Assess whether this behavior is consistent with proper packet-switched network design**, and explain the correct expected behavior. [10 marks]

Model Answer:

This behavior is **not** consistent with proper design – it violates the **store-and-forward rule**. A router must fully receive an entire packet (buffering it) and verify it before forwarding *any* bit of it onward; it should never forward partial packet data, since a receiver reconstructing a packet from disjoint, interrupted fragments has no reliable way to validate correctness, and downstream routers rely on each packet being a complete, self-contained unit. The **correct** behavior: if the link fails mid-transmission before the full packet is received, the router should discard the incomplete packet entirely. This will surface later as packet loss, detected and corrected by TCP retransmission at the Transport layer – not silently patched together at the router.

Q8. A satellite ISP advertises “1 Gbps ultra-fast internet” for a remote rural area, but users report noticeable lag (200+ ms) during real-time voice calls despite fast file downloads. **Explain this apparent contradiction** using the correct delay formulas. [5 marks]

Model Answer:

This is the classic **bandwidth $\neq$ latency** case. File downloads are throughput-bound (they benefit from the 1 Gbps rate reducing *transmission* delay, L/R , for large transfers), but voice calls are latency-sensitive and dominated by **propagation delay** (d/s), which depends only on the physical distance to the satellite and back – unaffected by how high the bandwidth is. A geostationary satellite’s round trip covers roughly 71,500 km, producing ~ 240 ms of propagation delay regardless of link speed, which is perceptible as lag in real-time voice/video but largely invisible in a bulk file download.

Q9. A remote-work company notices that employees connecting from home offices in different countries experience wildly different video-call quality, even though all have “high-speed” connections advertised at 200+ Mbps. Some employees are 50 km from the nearest company data center; others are 9,000 km away. **Explain, using the correct delay formula, why physical location – not advertised bandwidth – is the dominant factor here**, and propose one infrastructure-level solution. [10 marks]

Model Answer:

Video calls are highly sensitive to **propagation delay** ($d_{prop} = d/s$), which depends only on physical distance and signal speed, not on bandwidth. An employee 9,000 km away will experience substantially more propagation delay (potentially tens of milliseconds more, plus additional hops/routers along the longer path) than one 50 km away, *regardless* of both having 200+ Mbps connections – since bandwidth only affects transmission delay (L/R), which is comparatively small and roughly equal for both once their access links exceed a modest threshold.

Solution: Deploy regional **Edge Locations / points-of-presence** (or additional regional data centers) closer to distant employee clusters, so video-call traffic is routed through infrastructure physically nearer to each user, directly reducing d in the propagation-delay formula for the majority of the call’s path.

Section 3 – Protocol Layers (OSI & TCP/IP)

Q10. A security researcher discovers that a rogue router on a corporate network is inspecting and logging the full HTTP URLs and JSON bodies of every packet it forwards – something a standard Layer 3 router should have no ability or need to do. **Explain, using the principles of protocol layering, why this represents both a design anomaly and a security concern.** [10 marks]

Model Answer:

Design anomaly: A standard router operates at **Layer 3**, making forwarding decisions using only IP addressing information; it is designed to remain unaware of Layer 7 (Application) content such as HTTP URLs or JSON bodies. For this router to inspect and log that content, it must be deliberately performing **deep packet inspection** far beyond its expected layer of operation – violating the core layering principle that each layer should interact only with its own header, not payload belonging to layers above it.

Security concern: This means the router is capturing potentially sensitive application data (URLs revealing browsing patterns, JSON bodies possibly containing personal/financial information) that should be opaque to a network-layer device – especially serious if this traffic is meant to be protected by TLS/HTTPS, in which case such inspection may indicate either unencrypted (insecure) traffic or an active TLS-interception/MITM compromise on the network.

Q11. A developer is troubleshooting a bug where data corruption appears to occur specifically when packets cross from a company’s internal Ethernet LAN onto a WiFi segment, but the data is fine once it re-enters a wired LAN further downstream. **Using the encapsulation/decapsulation model, identify which layer(s) are most likely involved and explain your reasoning.** [10 marks]

Model Answer:

The most likely culprit is the **Data Link layer (Layer 2)**. Each time a packet crosses from one physical medium/technology to another (Ethernet → WiFi → Ethernet again), its Layer 2 frame is **decapsulated and re-encapsulated** with a new frame appropriate to that medium (different MAC addressing/framing rules for Ethernet vs. WiFi) – this happens at every hop, unlike the Layer 3 (IP) addressing, which largely persists end-to-end. If corruption appears specifically at the Ethernet-to-WiFi transition and resolves once back on wired Ethernet, this strongly suggests an issue in how the intermediate device re-frames/re-encapsulates data for the WiFi segment specifically (e.g., faulty Layer 2 error-checking or a misconfigured access point) – not a Layer 3 (IP) or Layer 4 (Transport) issue, since the IP payload itself should be untouched by this framing/re-framing process if done correctly.

Q12. A student argues: “OSI and TCP/IP are competing standards, and only one of them is actually used by real devices today.” **Assess this statement and correct any inaccuracies.** [5 marks]

Model Answer:

This is inaccurate. OSI and TCP/IP are **not competitors** – OSI (7 layers, developed by ISO) is primarily a **conceptual/teaching model** used to reason about and discuss networking concepts, while TCP/IP (4 layers, developed by DARPA) is the model that **actual Internet infrastructure and devices run on**. In practice, engineers commonly use OSI terminology (e.g., “that’s a Layer 4 issue”) even while the real traffic is flowing through TCP/IP’s 4-layer architecture – the two models coexist as complementary tools rather than as rival, mutually exclusive implementations.

Q13. A network device manufacturer claims their new “Layer 2.5 Smart Switch” can make forwarding decisions using IP addresses IN ADDITION to MAC addresses, while still being marketed purely as a

switch. **Explain what this would mean for the device’s classification** within the OSI device-layer framework, and whether such hybrid devices are plausible. [10 marks]

Model Answer:

If a device makes forwarding decisions based on **both** MAC addresses (Layer 2) and IP addresses (Layer 3), it is functionally behaving as a **Layer 3 device** (or at minimum a "Layer 2/3" multilayer device) for at least part of its operation – the OSI device table maps "decision basis" to layer, and using IP addresses for routing decisions is a Layer 3 function regardless of marketing terminology. Such hybrid devices ARE plausible and common in real networks (often called "Layer 3 switches"), which combine fast Layer 2 switching within a LAN with limited Layer 3 routing capability between VLANs/subnets – the key exam takeaway is that a device’s *true* OSI layer classification is determined by what information it actually uses to make decisions, not by its product name.

Section 4 – Devices, Topologies & Cloud Infrastructure

Q14. A 200-person company’s IT team is redesigning their network. Currently, every department uses a hub, causing constant congestion. Management wants: (a) fast local communication within each of 5 departments, (b) resilience so that no single device failure can take down more than one department, and (c) manageable cost. **Propose a complete device and topology solution**, justifying each choice with reference to OSI layers and topology trade-offs. [15 marks]

Model Answer:

Step 1 – Replace hubs with switches (Layer 2) within each department. This immediately eliminates the unnecessary broadcast congestion (a hub blindly repeats to every port; a switch learns MAC-to-port mappings and forwards only to the intended device), satisfying goal (a).

Step 2 – Use Star topology within each department, with each department’s devices cabled to its own dedicated switch. This is easy to set up/debug and appropriate for department-sized groups (per the small-office topology recommendation), and confines the switch’s single-point-of-failure risk to just that one department – satisfying goal (b), since one department’s switch failing does not affect the other four departments.

Step 3 – Connect the five departmental switches to a central router (Layer 3). The router allows inter-department communication using IP addressing/routing, effectively forming a **Hybrid topology** (multiple Star sub-networks joined at a Layer 3 core) – this matches the notes’ recommendation that larger, more complex environments benefit from hybrid designs.

Step 4 – Cost control: Since only 5 switches + 1 router are needed (versus a full mesh of $200 \times 199/2$ direct connections), cabling cost stays linear in the number of devices, satisfying goal (c). A **firewall** could additionally be placed at the router/internet boundary to filter traffic by IP/port for security.

Q15. A network operator sees that their firewall is blocking an internal application’s traffic because the firewall only checks IP addresses and port numbers, but the actual malicious traffic is hidden inside a technically well-formed HTTPS packet on port 443 (the same port used by legitimate encrypted banking traffic). **Explain why the basic firewall cannot detect this threat**, and what device-layer capability would be required to do so. [10 marks]

Model Answer:

A **basic packet-filtering firewall** reads only two layers deep: **Layer 3** (source/ destination

IP) and **Layer 4** (source/destination port). Since the malicious traffic uses a legitimate-looking destination IP and the standard HTTPS port (443) – identical to legitimate encrypted banking traffic – the firewall has no way to distinguish malicious payload from benign payload using only IP+port criteria; the actual *content* is invisible to it (and further obscured by encryption).

To detect this, the network would need a more advanced device capable of deeper inspection – i.e., operating with awareness up toward **Layer 7 (Application layer)**, such as a next-generation firewall / intrusive detection system with TLS-termination or decryption capability, allowing it to inspect the actual application-layer content rather than just IP/port metadata. This illustrates the general trade-off: deeper layer awareness gives more security insight but at higher inspection cost/complexity.

Q16. A smart-home installer wires 8 IoT devices (bulbs, locks, cameras) so that every device has a direct wireless connection to every other device. A customer asks why the installer didn't simply run all devices through one central WiFi router instead. **Justify the installer's mesh design choice** using fault-tolerance and scalability trade-offs. **[5 marks]**

Model Answer:

The installer chose **Mesh topology** because smart-home IoT devices benefit specifically from **high fault tolerance**: if one device (or the path to it) fails, other devices can still communicate via alternate direct paths, keeping the smart-home system functional. A single central-router (Star) design would work for 8 devices, but the central router becomes a **single point of failure** – if it goes down, the entire smart home loses connectivity at once. The trade-off is more connections needed ($n(n-1)/2 = 28$ possible links for 8 devices, though wireless mesh doesn't require literal cabling for each), but for a small, reliability-sensitive IoT deployment, this is exactly the environment the notes recommend mesh for.

Q17. An online retailer's checkout server becomes unresponsive during a sale, even though monitoring shows the network links have plenty of unused bandwidth. Simultaneously, their network team wants to add MORE bandwidth to the data center's internet connection as the fix. **Evaluate whether this proposed fix addresses the actual bottleneck**, and propose the correct fix. **[10 marks]**

Model Answer:

The proposed fix (adding bandwidth) **does not** address the actual bottleneck. Since monitoring shows the network *links* have spare capacity, the bottleneck is not a link-rate problem (which bandwidth would help) but a **server-side** problem – the checkout server itself cannot process the volume of incoming requests fast enough (e.g., its request queue/thread pool is saturated), a scenario explicitly distinguished in the notes as a case where "the server is the bottleneck, not the link."

Correct fix: Add **more checkout servers** and deploy a **load balancer** (Layer 4/7) to distribute incoming checkout requests across them. This directly targets the true constraint (server processing capacity) rather than a network resource that was never actually scarce.

Q18. A university lab installs a new device that, when it receives an electrical signal on any one port, immediately re-transmits that identical signal out of every other port with no processing or addressing logic whatsoever. **Identify this device and its OSI layer**, and explain the specific downside students will notice as more computers are added to the lab. **[5 marks]**

Model Answer:

This is a **hub**, operating at **Layer 1 (Physical)**. Because it blindly broadcasts every incoming signal to all other ports with no MAC-address-based intelligence, every device on the network

receives traffic intended for every other device, forcing each to inspect and discard irrelevant frames. As more computers are added, the shared medium becomes increasingly congested (more devices all receiving all traffic), and collisions/contention for the shared broadcast domain increase – unlike a switch, which would only send traffic to the specific intended port.

Section 5 – HTTP, REST & Application Protocols

Q19. An API team designs an endpoint so that calling `PUT /api/profile/55` with a request body containing only `{"nickname": "Sam"}` is expected to update **ONLY** the nickname field, leaving all other profile fields untouched. After deployment, users report that other profile fields (bio, avatar, preferences) are being wiped out whenever they change their nickname. **Diagnose the root cause of this bug** and propose the correct fix using proper REST semantics. **[10 marks]**

Model Answer:

Root cause: The team is using **PUT** for what is actually a **partial update**. PUT's REST semantics mean "replace the entire resource with exactly what's in this request body" – since the request body only contains `nickname`, a standards-compliant server implementation interprets this as "the **ENTIRE** profile is now just `{nickname: Sam}`", wiping out bio, avatar, and preferences, which were simply absent from the request.

Fix: Switch the endpoint to use **PATCH** instead of PUT for this use case. PATCH is explicitly designed for partial updates – only the fields present in the request body are modified, and all other existing fields are left untouched. If PUT must be kept for full-profile replacement elsewhere, the API should support both methods with their correct distinct semantics rather than using PUT for partial updates.

Q20. A media streaming company notices that under HTTP/2, when one video segment's request experiences packet loss, **ALL** other in-flight requests (thumbnails, subtitles, next segment pre-fetch) on the same connection stall momentarily – even though HTTP/2 was specifically adopted to fix Head-of-Line blocking. **Explain why this happens despite using HTTP/2**, and **recommend a protocol-level change** that would resolve it, with justification. **[15 marks]**

Model Answer:

Why it happens: HTTP/2's **multiplexing** does fix *application-level* HoL blocking – multiple streams (video segment, thumbnails, subtitles) can be requested and served out of strict order over one connection. However, HTTP/2 still runs over a **single underlying TCP connection**, and TCP guarantees strictly in-order delivery of bytes on that connection. If a TCP segment carrying part of the video-segment stream is lost, TCP must pause delivery of *everything* behind it (including unrelated streams like thumbnails/subtitles) until the lost segment is retransmitted and received – this is **TCP-level Head-of-Line blocking**, a residual limitation of HTTP/2 that multiplexing alone cannot fix.

Recommendation: Migrate to **HTTP/3**, which runs over **QUIC (UDP-based)** instead of TCP. QUIC gives each stream genuinely independent delivery guarantees at the transport level, so packet loss affecting the video-segment stream would only delay that specific stream – thumbnails, subtitles, and pre-fetch requests would continue delivering normally, fully resolving the residual stalling HTTP/2 could not.

Q21. A backend engineer wants to allow API clients to safely retry a request automatically if no response is received (e.g., due to a timeout), without risking duplicate side effects like creating two orders instead of one. **Which HTTP method(s) would be safe to auto-retry without this**

risk, and which would NOT be, in general REST design?

[5 marks]

Model Answer:

Methods considered "safe" (no side effects) – **GET, HEAD, and OPTIONS** – can generally be retried freely, since repeating them simply re-reads data without changing server state. Methods with side effects – particularly **POST** (Create) – are risky to blindly auto-retry, since a lost response doesn't necessarily mean the original request failed to reach the server; retrying could create a *second* order. PUT and DELETE are comparatively safer to retry than POST because they are typically **idempotent** (repeating them produces the same end state), but this still depends on correct server-side implementation – POST remains the riskiest method to naively auto-retry without additional safeguards like idempotency keys.

Q22. A client application repeatedly sends the exact same HTTP GET request to an API every 2 seconds to "check for updates," even though the underlying data changes only once every few hours. The server team is concerned about unnecessary load. **Using concepts from the HTTP lecture (not just "add caching" generically), explain what specific HTTP mechanism(s) could reduce this load, and how they would work here.**

[10 marks]

Model Answer:

Because GET is inherently **stateless and cacheable**, the server can use HTTP's caching and conditional-request headers to avoid reprocessing identical requests. Specifically: (1) **Response caching headers** (e.g., Cache-Control, Expires) tell the client it can safely reuse a previously fetched response for a defined period rather than re-requesting immediately. (2) **Conditional GET** using an ETag or Last-Modified header lets the client send a lightweight "has this changed since X?" check; if unchanged, the server can reply with a **304 Not Modified** status code (from the 3xx redirection/no-change range covered in the notes) with an empty body instead of resending the full payload – dramatically reducing bandwidth and server processing for data that rarely changes, while still letting the client detect updates promptly.

Q23. A budget airline's booking website uses HTTP/1.0-style behavior: every image, CSS file, and script on the booking page triggers a brand-new TCP connection. During a high-traffic sale, the web servers report exhausting available connection resources despite moderate actual data volume. **Diagnose why connection exhaustion (not bandwidth) is the bottleneck here, and recommend a specific HTTP-version-based fix.**

[10 marks]

Model Answer:

Diagnosis: Since every single resource (each image, the CSS file, each script) opens a **brand-new TCP connection** (classic HTTP/1.0 behavior – no persistent connections), the server must perform a full TCP handshake and allocate connection-tracking resources for every individual file on every page load. Under high traffic, the sheer *number* of simultaneous connections being opened/torn down exhausts server-side connection resources (e.g., file descriptors, memory for connection state) long before the actual data volume becomes a bandwidth problem – the bottleneck is connection *overhead*, not payload size.

Fix: Upgrade to **HTTP/1.1 at minimum** (Keep-Alive persistent connections, so one TCP connection serves multiple requests) or, better, **HTTP/2**, which multiplexes many resource requests over a *single* TCP connection per client – dramatically reducing the number of concurrent connections the server must manage while serving the same number of resources.

Q24. A gaming company's matchmaking API uses GET requests to actually create new matchmaking sessions (i.e., calling GET /api/matchmaking triggers a new match to be created server-side as a side effect), reasoning that "GET is simpler to call from the client." A code reviewer flags this as a serious

REST design violation. **Justify the reviewer's concern** and explain a realistic failure mode this design could cause in production. [10 marks]

Model Answer:

The reviewer is correct: GET is defined as a **safe method** – by REST/HTTP convention it must have **no side effects** and must not alter server state. Using GET to trigger match creation violates this fundamental assumption relied upon throughout the web ecosystem (browsers, proxies, CDNs, crawlers, and HTTP/1.1 pipelining logic all assume GET requests can be freely repeated, cached, or pre-fetched without consequence).

Realistic failure mode: A CDN or browser could **pre-fetch** or **cache** the GET request (since GET responses are typically cacheable), or a network issue could cause the client/browser to silently retry the "safe" GET request – either of which would spawn **duplicate matchmaking sessions** the user never intended to create, potentially breaking matchmaking state, wasting server resources, or placing the same player into multiple simultaneous matches. The correct design would use **POST** (Create), which is explicitly expected to have side effects and is never assumed to be safely cacheable/retriable by intermediaries.

Section 6 – HTTPS, TLS & Network Security

Q25. A fintech startup's security auditor flags that although their website shows a valid padlock icon, a penetration test revealed the site was serving a phishing page indistinguishable from the real login page, hosted on a similarly-spelled domain (e.g., "faintech.com" instead of "fintech.com") that ALSO has a valid HTTPS certificate from a legitimate CA. **Explain why the padlock icon did not prevent this attack**, and describe what additional practice (beyond relying on HTTPS alone) users and the company should adopt. [15 marks]

Model Answer:

Why the padlock didn't help: The padlock only guarantees two things – the connection is **encrypted** and the server's **identity has been verified by a CA for the domain name presented**. It does **not** guarantee the domain itself is legitimate/trustworthy or free of malicious intent. A CA will happily issue a valid certificate for "faintech.com" because that domain's owner (the attacker) legitimately controls it – the CA has no way (and no responsibility) to judge whether the domain name is deceptively similar to a real brand. This is exactly the "microsoft.com vs. a look-alike domain" trap flagged in the notes.

Additional practices: Users should carefully verify the **exact domain name** in the address bar rather than just trusting the padlock; the company should register common typo/homoglyph domain variants defensively, monitor for phishing look-alikes, consider Extended Validation certificates or domain-monitoring services, and educate users that "HTTPS" and "padlock" mean "encrypted + this specific domain's identity is verified," never "this site is trustworthy."

Q26. During a TLS handshake, a server accidentally sends its certificate BEFORE completing the asymmetric key exchange, and a junior engineer proposes "skipping straight to symmetric encryption immediately using a hardcoded shared key to save time," bypassing the handshake steps entirely. **Explain the security flaws in this proposal**, referencing the actual purpose of each TLS phase. [10 marks]

Model Answer:

This proposal breaks TLS's core security model in multiple ways:

(1) **No authentication:** Skipping certificate validation means the client never confirms it is actually talking to the legitimate server rather than an impersonator/MITM attacker – defeating

the *Authentication* goal of cryptography entirely.

(2) **No secure key establishment:** A "hardcoded shared key" reintroduces exactly the **key-distribution problem** that asymmetric cryptography exists to solve – if the same static symmetric key is embedded/shared out-of-band, an attacker who ever obtains it (e.g., via reverse-engineering the client) can decrypt ALL past and future traffic, and there's no way to safely establish a unique key per session with strangers.

(3) **Loss of forward secrecy:** Proper TLS handshakes typically derive a session key fresh per session; a hardcoded key means a single compromise exposes every session ever conducted, past and future, rather than just one session.

The correct process – ClientHello → ServerHello+Certificate → Certificate Validation → Key Exchange → Encrypted Communication – exists precisely to establish both **verified identity** and a **freshly and safely negotiated** session key before any real data is exchanged.

Q27. A company decides to encrypt ALL of its internal application data using ONLY asymmetric encryption end-to-end (no symmetric encryption at all) for "maximum security," even for large file transfers between employees. **Critically assess this design decision** and explain what TLS's actual approach reveals about its flaws. **[10 marks]**

Model Answer:

This design decision is **poorly justified**. While asymmetric encryption is genuinely strong for establishing trust/keys between parties who've never securely shared a secret before, it is **computationally much slower** than symmetric encryption, making it impractical for encrypting large volumes of data (e.g., large file transfers) directly – performance would degrade severely at scale. TLS's actual design reflects the better approach: use **asymmetric** cryptography only briefly, during the handshake, to safely establish a shared **symmetric session key**, then switch to **fast symmetric encryption** for all the actual bulk data. The company should adopt this same "best of both worlds" hybrid approach – using asymmetric encryption for secure key exchange/authentication and symmetric encryption for the bulk file data itself – rather than asymmetric encryption throughout.

Q28. A user notices their browser displays "Connection is Not Secure" when visiting an internal company tool that uses a self-signed certificate generated by the IT department (not from a public CA). **Explain, from a chain-of-trust perspective, why this warning appears**, even though the connection may genuinely be safe within the company network. **[5 marks]**

Model Answer:

The warning appears because the browser's **certificate validation** process checks whether the presented certificate is signed by a **CA that the browser/OS already trusts** (via its pre-installed root CA list). A self-signed certificate is signed by the issuer itself (the IT department), not by any recognized root CA, so it fails this chain-of-trust check – *regardless* of whether the connection is actually safe in practice. This is a deliberate, conservative security default: browsers cannot distinguish "safe internal self-signed cert" from "malicious self-signed cert used by an attacker" using automated validation alone, so both trigger the same warning. (In practice, companies can resolve this by installing their own internal root CA certificate on employee devices, or using a certificate from a recognized CA.)

Q29. An attacker manages to compromise a Certificate Authority and issues themselves a fraudulent but technically valid (correctly signed) certificate for "yourbank.com." **Explain why this scenario is considered one of the most severe failures of the HTTPS trust model**, and what this implies about the limits of certificate-based authentication. **[10 marks]**

Model Answer:

This scenario is severe because it attacks the **foundation** the entire chain-of-trust model relies on: browsers trust certificates *because* they are signed by a CA that is itself already trusted. If a CA is compromised, an attacker can issue certificates that will pass full, correct certificate validation – the browser has no way to detect anything wrong, since the signature genuinely traces back to a trusted root. This means the attacker could impersonate "yourbank.com" in a MITM attack, and standard TLS defenses (which rely entirely on catching *invalid/untrusted* signatures) would not flag it. This illustrates that certificate-based authentication is only as strong as the **security of the CAs themselves** – it does not eliminate trust, it *concentrates* trust into a small number of highly sensitive organizations, which is why CA security practices, certificate transparency logs, and revocation mechanisms are critical (though beyond this course's core scope) additional safeguards in real-world HTTPS infrastructure.

Q30. A hospital's telemedicine platform must guarantee that video consultations are never intercepted or silently modified, even by staff on the hospital's own internal network. An engineer proposes using HTTP with a custom XOR-based obfuscation scheme instead of HTTPS/TLS, arguing "it's faster and good enough." **Critically evaluate this proposal** against the three core goals of cryptography. **[10 marks]**

Model Answer:

The proposal fails against all three cryptographic goals:

Confidentiality: A simple XOR obfuscation scheme (especially with any fixed/short key) is trivially reversible by anyone with basic cryptanalysis skills or network access – unlike TLS's use of properly vetted symmetric algorithms (e.g., AES) with securely established session keys, it offers negligible real protection against eavesdropping.

Integrity: Without a Message Authentication Code or hashing mechanism, there is no way to detect if consultation data (video/audio streams, patient notes) has been tampered with in transit – an attacker (or malicious insider) could alter data undetected.

Authentication: Without TLS's certificate-based server identity verification, there is no cryptographic guarantee that the client is actually talking to the legitimate telemedicine server and not an impersonator, even on the "internal" network (insider threats or a compromised internal device could still perform MITM).

Verdict: The proposal should be rejected. Proper **HTTPS/TLS** (with strong, industry-standard ciphers) is necessary – "faster" is not an acceptable trade-off against patient data confidentiality, integrity, and authenticity, particularly in a regulated healthcare context.

Q31. A small business owner asks: "If I get an HTTPS certificate and the green padlock shows up on my checkout page, does that mean customer payment data is fully secure end-to-end, including inside my own servers and databases?" **Correct this misunderstanding** clearly and concisely. **[5 marks]**

Model Answer:

No – this is a common but important misconception. HTTPS/TLS only secures the data **in transit**, i.e., *while it travels between the customer's browser and the business's server* (encryption + server identity verification). It says **nothing** about what happens to that data once it **arrives** at the server: if the server logs card numbers in plaintext, stores them insecurely in a database, or has weak internal access controls, that data can still be exposed, regardless of a perfectly valid HTTPS padlock on the checkout page. Full security requires additional measures beyond TLS – e.g., not storing raw card data at all (using a PCI-compliant payment processor/tokenization), encrypting sensitive data at rest, and securing server/database access – the padlock covers only the "in-transit" leg of the data's journey.

Exam Strategy Notes

- **Answer the command word exactly.** "Explain" wants mechanism/reasoning; "Diagnose" wants root-cause identification; "Critically evaluate" wants you to state a verdict (accurate/inaccurate, correct/flawed) AND justify it – don't just describe the scenario back.
- **Name the concept explicitly.** Examiners reward you for naming the exact principle (e.g., "store-and-forward rule," "TCP-level Head-of-Line blocking," "chain of trust") rather than describing it vaguely.
- **For calculation-based scenarios,** always show the formula, the substituted numbers, and the final answer with correct units – partial credit is usually given for correct method even with an arithmetic slip.
- **For "propose a fix" questions,** structure your answer as: root cause → why the naive/wrong fix fails → the correct fix → why it specifically resolves the stated problem.
- **Cross-lecture scenarios are common** – a single question may need you to combine delay/throughput concepts (Lecture 2) with devices (Lecture 4) or HTTP (Lecture 5) with TLS (Lecture 6). Don't answer from only one lecture's toolbox if the scenario clearly spans more.

All the best for your test – CSAI321 Computer Networks.